Latent Feature Extraction for Musical Genres from Raw Audio: Reproduction and Improvements

UE24CS352A Machine Learning — Mini-Project Write-up · Team: Harsha K, Sunil
Code: github.com/Harsha-Kamaraj/ML-Mini-Project (notebook *musical_genre_paper_reproduction.ipynb*)

1. Problem statement

The goal is to recognise the genre of a short music clip (classical, jazz, metal or pop) and to learn a compact genre *embedding*, directly from the raw audio waveform and without hand-crafted audio features. The task has two parts. (a) **Reproduce** Sawhney, Vasavada & Wang, *Latent Feature Extraction for Musical Genres from Raw Audio* [1]. Its Deep Softmax Autoencoder (an autoencoder with a softmax classifier on its 64-dim bottleneck) reports 65.3% test accuracy. (b) **Build models on top of it** that improve accuracy, and evaluate them fairly.

2. Dataset

We use the **GTZAN Genre Collection** [2]: 1,000 thirty-second mono recordings at 22,050 Hz in 10 genres, 100 per genre. Following the paper, we use 4 genres × 100 = **400 recordings**. The original host has been offline since 2022, so we downloaded the audio from a Hugging Face mirror. We verified its checksum and decoded every file in full: all 400 are readable, 29.9–30.6 s long. For the selected genres, the audio matched a second, independent mirror sample for sample on all 399 files that copy could read (its copy of *jazz.00054* is corrupt). 18 recordings are exact duplicates of other recordings; we keep each duplicate in the same partition as its original. **Preprocessing** follows the paper: the files are converted to .wav and loaded with LibROSA; each song is cut into 20 consecutive segments of 20,000 samples (0.907 s); each segment is average-pooled with a window of 40 to a 500-dim vector. This gives **8,000 examples** (2,000 per genre), randomly split 6,000 / 1,000 / 1,000 into train / dev / test, with no normalisation.

3. Approach

3.1 Reproduction

We implemented the paper's three models in PyTorch:

- a **two-layer NN** (500 → 128 tanh → 4, cross-entropy);
- a **vanilla autoencoder** (500-256-192-128-**64**-128-192-256-500, loss $||x - g(f(x))||^2$);
- the **Deep Softmax AE**: the same autoencoder plus a 64 → 32 → 16 → 4 classifier, with loss = $\gamma||x - g(f(x))||^2 + (1 - \gamma)\cdot$CE and $\gamma = 0.9$.

Training uses Adam (learning rate 10^{-4}, batch size 512) and dropout with keep probability 0.9. Rule for each setting: **the paper first**; where it is silent (activations, dropout placement, initialisation, epochs), the authors' linked code; where the two conflict, the paper. An appendix tests why our numbers differ from the paper's.

3.2 Improvements (one change per step, so each gain is measured separately)

- **Step 1 — Fix the model.** The paper's ReLU decoder output cannot produce negative amplitudes; in training it collapses to 0 for every input. We use a linear output layer, and add input standardisation, reshuffling every epoch, best-dev checkpointing with early stopping, and a re-tuned γ.
- **Step 2 — More signal.** Smaller pooling windows were tried (dev kept the paper's 40), and the remaining 13 segments of each song were added to the training data (chosen by dev).
- **Step 3 — Convolutional Deep Softmax AE on raw audio (main contribution).** Encoder: four strided 1-D convolution blocks (16→128 channels, batch norm, ReLU), then global average + max pooling, giving a **64-dim encoding**. Decoder: transposed convolutions. Head: the paper's 64→32→16→4. Adam, learning rate 10^{-3}, batch size 64.
- **Step 4 — Mel-spectrogram Deep Softmax AE (extra).** The same design with 2-D convolutions on 64-band log-mel spectrograms (64 × 40). This is the feature-engineering direction the paper lists as future work.

3.3 Evaluation protocol

The paper's split shuffles individual clips, so clips of the same song appear in both train and test (362 of 400 songs in our split). We therefore report two splits:

- the **clip split** (the paper's), for comparison with the paper;
- a **song split** (320 / 40 / 40 songs; 80 / 10 / 10 per genre; duplicates grouped), so that every test song is unseen.

Every model is trained with **5 seeds** on both splits and reported as mean ± std. Every hyperparameter (γ, pooling, extra segments) is chosen automatically by a sweep on the **dev** set; the test set is only used for reporting. Metrics: clip accuracy, macro-F1, and song-level accuracy (the mean softmax over a song's test clips).

4. Implementation

Everything is in one self-contained Jupyter notebook (Python 3.14, PyTorch 2.14, LibROSA 1.0, scikit-learn). **Part 1** covers data acquisition and checks, preprocessing, the three paper models, Tables 1–2 and Figures 1, 3, 4, 5 and 6, and the comparison appendix. **Part 2** contains the split builder, a shared trainer and evaluator, the four improvement steps with their dev sweeps, a final comparison table, best-model analysis (confusion matrix, PCA of the 64-dim encodings and 4-dim logits) and a live demo cell. Training runs on CPU with fixed seeds for repeatability. Results and weights are cached under a hash of each configuration: *Run All* takes about 4 min with the cache and about 2–3 h from scratch on a laptop. Assertions check data shapes, class balance, split disjointness and duplicate grouping.

5. Results

Model	Clip split test acc.	Song split test acc.	Song split song-level acc.
Paper — two-layer NN (quoted)	36.4%	—	—
Paper — Deep Softmax AE (quoted)	65.3%	—	—
Our reproduction — two-layer NN	39.0 ± 1.4%	38.1 ± 2.9%	46.2 ± 9.2%
Our reproduction — Deep Softmax AE	61.0 ± 2.2%	58.4 ± 3.1%	70.9 ± 7.4%
Step 1: fixed Deep Softmax AE	61.5 ± 1.2%	60.3 ± 3.2%	70.8 ± 8.8%
Step 2: + more signal	64.3 ± 0.6%	62.9 ± 2.3%	71.3 ± 8.2%
Step 3: Conv Deep Softmax AE (raw audio)	**87.2 ± 1.7%**	**83.2 ± 2.9%**	**91.0 ± 4.9%**
Step 4: Mel Deep Softmax AE (extra)	97.7 ± 0.8%	94.5 ± 2.9%	97.0 ± 2.8%

Table 1: Test accuracy, mean ± std over 5 seeds. Quoted paper values come from [1]; every other value was computed by our notebook.

Reproduction. The baseline matches the paper (39.0% vs 36.4%), and so does the Deep Softmax AE's train accuracy (94.8% vs 94.9%). Its test accuracy is lower: 61.0% (range 57.7–63.9% across seeds) vs 65.3%. The appendix tested three causes:

- the unreported split and seed alone move test accuracy by about 6 points;
- the authors' released code settings reach only 85.4% train accuracy, not the reported 94.9%, so the published table probably did not come from that code;
- the decoder's ReLU output dies (0% non-zero outputs), so the reconstruction term is a constant and the Deep Softmax AE trains as a plain classifier.

Improvements. Fixing the bugs (Step 1) and adding signal (Step 2) gave at most +3 points. Replacing the fully connected encoder with convolutions on the same raw audio (Step 3) raised clip-split test accuracy to **87.2%**, about 22 points above the paper, and reached **83.2%** on unseen songs. The mel-spectrogram model (Step 4) is best overall at 97.7% / 94.5%; its per-genre test F1 is 0.96–0.99, against 0.53–0.78 in the paper. In its PCA plots the four genres form clearly separated clusters.

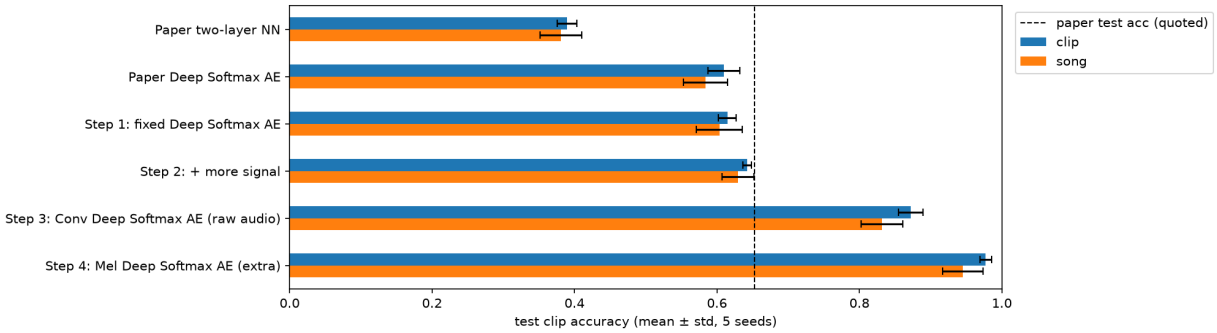


Figure 1: Test accuracy of every model on the clip split (paper protocol) and the song split (unseen songs). The dashed line is the paper's reported 65.3%.

6. Conclusions

- The reproduction is faithful for the baseline. The Deep Softmax AE gap is explained by an unreported split, seed variance, and a mismatch between the paper and its released code. We report 5-seed ranges, not a single run.
- The paper's reconstruction objective was effectively disabled by a dying ReLU output. Once fixed, γ chosen on dev made no measurable difference in the convolutional models (within 0.2 points): the gains come from architecture and input representation, not from the autoencoder objective.
- **Architecture is the main lever.** Convolutions on raw audio improve on the paper by about 22 points. Spectrograms add about 10 more, at the cost of the paper's "no feature engineering" premise.
- A clip-level split overstates performance by 1–4 points compared with unseen songs. Song-disjoint evaluation should be the standard.

Limitations. GTZAN is small (400 songs; 40 test songs per seed). Artist overlap is not controlled, because GTZAN has no artist labels and repeats some artists [3]. Tuning was limited by CPU-only training.

References. [1] A. Sawhney, V. Vasavada, W. Wang. *Latent Feature Extraction for Musical Genres from Raw Audio*. Stanford University, 2018. [2] G. Tzanetakis, P. Cook. *Musical Genre Classification of Audio Signals*. IEEE Trans. Speech and Audio Processing, 2002. [3] B. L. Sturm. *The GTZAN dataset: Its contents, its faults, their effects on evaluation, and its future use*. arXiv:1306.1461, 2013.